

How HomeVault is built

Technology stack, approach, key decisions, challenges and the deployment process

Next.js · NestJS · PostgreSQL · Prisma

The project at a glance

12

backend modules

NestJS, one per domain

10

database tables

PostgreSQL via Prisma

46

API routes

REST + JSON, JWT-secured

12

end-to-end flows

tested in a real browser

What was delivered

- A complete web app: public website + signed-in household workspace
- Separate frontend and backend projects talking over a REST API
- Docker setup for the database, demo data, and a step-by-step README
- A free-tier hosting plan (Vercel + Render + Neon + Cloudinary) with a ready Render blueprint

Technology stack

Frontend

- Next.js 15 (App Router)
- React 19 + TypeScript
- Tailwind CSS 4
- Recharts (charts)
- Lucide icons

Why: Fast, SEO-friendly landing page and a rich app in one codebase; first-class Vercel hosting.

Backend

- NestJS 11 + TypeScript
- Passport JWT authentication
- class-validator DTOs
- Multer file uploads
- bcrypt password hashing

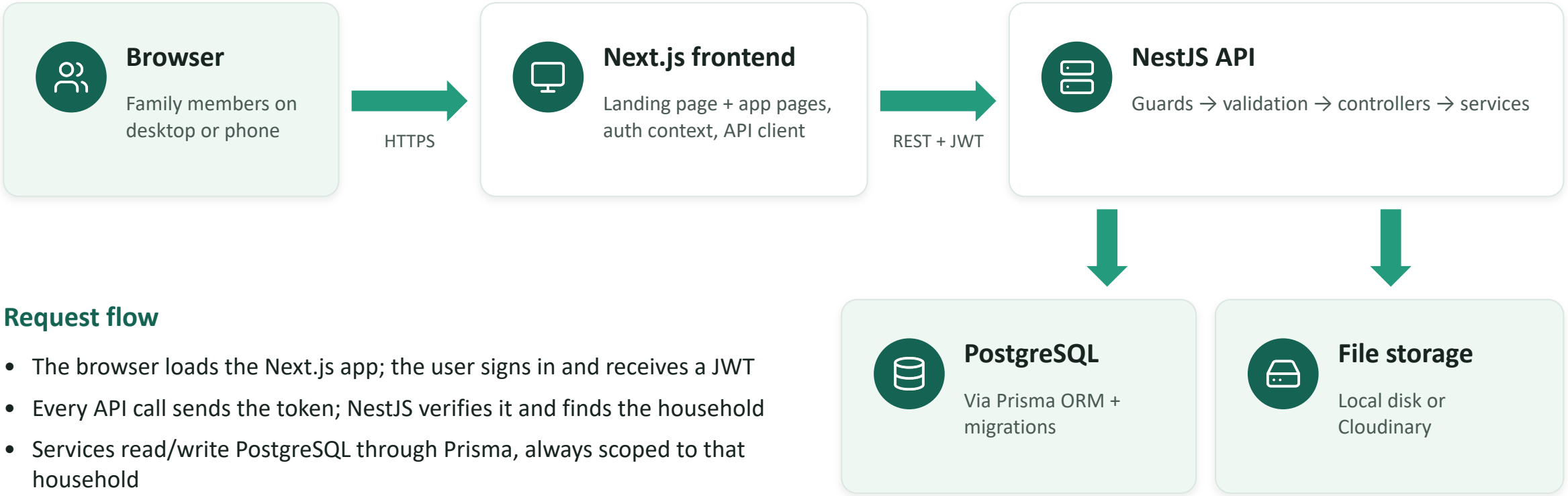
Why: Opinionated module structure, dependency injection and guards keep a growing API organized.

Database

- PostgreSQL 16
- Prisma 6 ORM
- Versioned SQL migrations
- Seed script for demo data
- Docker Compose locally

Why: Household data is highly relational; Postgres is reliable, free to host and Prisma gives type-safe queries.

System architecture



Request flow

- The browser loads the Next.js app; the user signs in and receives a JWT
- Every API call sends the token; NestJS verifies it and finds the household
- Services read/write PostgreSQL through Prisma, always scoped to that household
- Uploaded files go to storage; only their URL is saved in the database

How it is built

Frontend, backend, database, API and security in detail

Frontend: Next.js 15 + React 19

frontend/src

```
app/page.tsx            landing page
app/login, register     auth pages
app/app/layout.tsx      shell + auth guard
app/app/*/page.tsx      10 module pages
components/ui.tsx       UI kit + forms
components/charts.tsx   spending charts
lib/api.ts              fetch client + token
lib/auth.tsx            auth context
lib/use-crud.ts         CRUD page hook
lib/format.ts           dates, money, labels
```

Auth context

Stores the JWT, loads the current user and redirects signed-out visitors to /login.

One API client

Adds the token, parses errors into readable messages and handles expired sessions.

Config-driven forms

One EntityForm renders every create/edit form from a field list, which keeps 6 CRUD pages small.

Design tokens

Brand colors and fonts defined once in Tailwind 4 @theme; consistent cards, badges and modals.

Backend: NestJS 11, one module per domain

Every request passes through the same pipeline



Modules

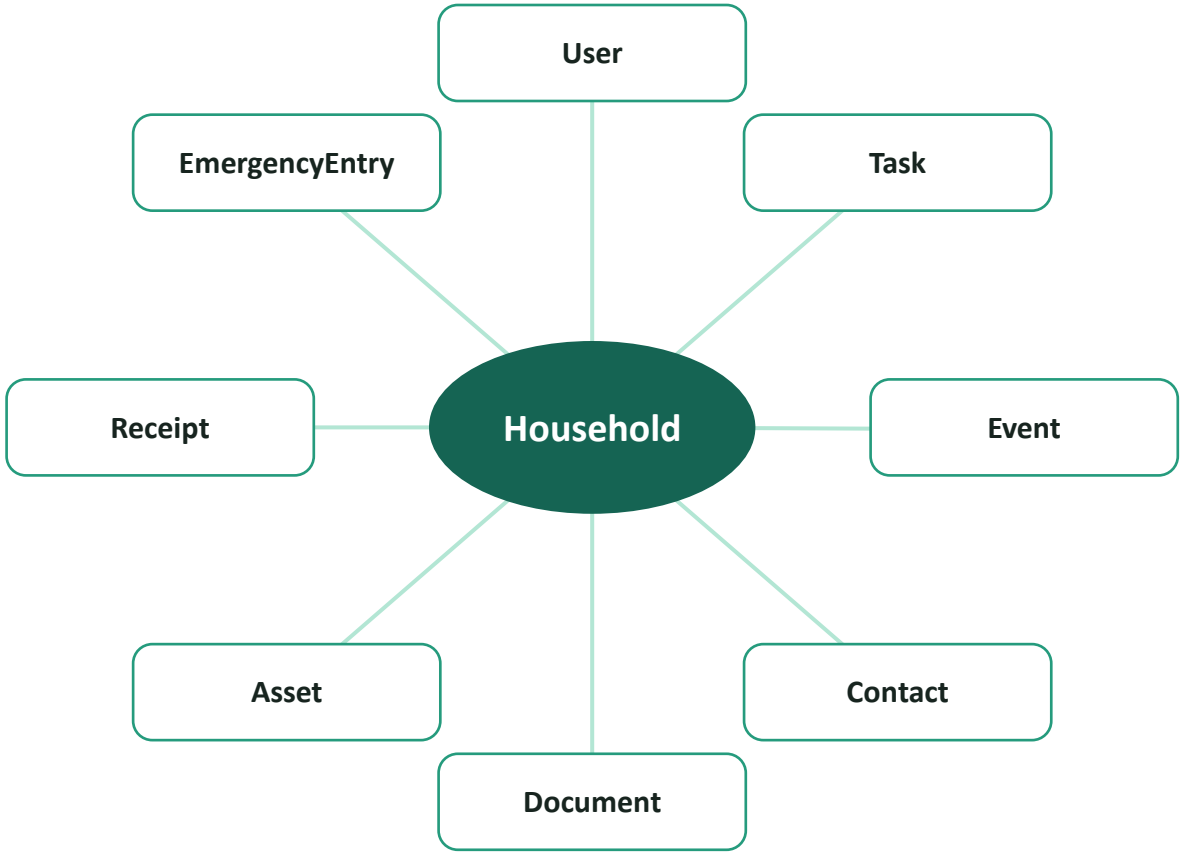
`auth`	`household`	`tasks`	`events`	`contacts`	`documents`
`assets`	`receipts`	`emergency`	`dashboard`	`storage`	`prisma`

Each module = controller + service + DTOs + module file. Adding or removing a feature touches one folder.

Patterns that keep it maintainable

- HouseholdScopedService: one base class gives every resource safe CRUD
- Global Prisma, Storage modules injected where needed
- AdminGuard protects member management
- Health check at /api/health for hosting

Database: PostgreSQL + Prisma



ReceiptItem belongs to Receipt; Asset can link to its Receipt; Task can be assigned to a User.

Design choices

- Household is the tenant: every table carries householdId with an index
- Cascade deletes: removing a household removes all its data
- Enums for fixed states (Role, Plan, TaskStatus, Priority)
- Categories stored as validated strings, easy to extend
- Money as Float for simplicity (Decimal is a future upgrade)
- Schema changes ship as versioned SQL migrations

Why PostgreSQL over MongoDB? The data is relational (receipts → items → assets, users → tasks) and needs joins, constraints and transactions.

REST API design

Area	Endpoints (all under /api)	Notes
Auth	POST auth/register · POST auth/login · GET auth/me	Public except /me; returns a JWT
Household	GET/PATCH household · POST/DELETE household/members	Changes need Owner/Admin
Core CRUD	tasks · events · contacts · documents · assets · emergency	GET, POST, PATCH /:id, DELETE /:id
Receipts	GET/POST receipts · PATCH/DELETE receipts/:id · GET receipts/analytics	Analytics by month, category, HSA
Aggregates	GET dashboard · GET emergency/binder	One call per screen
Files	POST uploads (multipart)	10 MB limit, type allow-list
Ops	GET health	Used by Render health checks

Consistent rules: JSON in/out, validated DTOs, 400 with readable messages, 401 for bad tokens, 404 when a record isn't in your household.

Security approach

Passwords

Hashed with bcrypt (10 rounds); never stored or returned in plain text.

Sessions

Signed JWTs with a 7-day expiry; the user is re-checked on every request so removed members lose access.

Data isolation

Every query is filtered by householdId through a shared base service. No cross-family access.

Roles

Owner / Admin / Member; a server-side guard protects member and plan changes.

Validation

Whitelisted DTOs reject unknown fields; uploads limited by size and file type.

CORS & HTTPS

API only accepts the configured frontend origin; hosting providers enforce HTTPS.

Approach & challenges

How the work was planned, the key decisions, and the problems solved along the way

How I approached the build



1

Study the reference

Mapped the reference site’s sections, modules and pricing into a feature list.

2

Scope & stack

Agreed an MVP (website + working app) and the stack: Next.js, NestJS, Postgres.

3

Data model first

Designed the Prisma schema around the household as the tenant.

4

API module by module

Built and smoke-tested each module with real HTTP calls.

5

UI on shared parts

Built a small component kit first, then each page on top of it.

6

Test like a user

Browser tests of every flow + a clean install from the zip.

7

Iterate

Removed AI features on request, cleanly, via module boundaries.

Principle: get one thin slice working end to end (DB → API → UI), verify it, then widen. Every step was checked before moving on.

Key decisions & trade-offs

Decision	Chosen	Alternatives	Why
Frontend framework	Next.js 15	Vite + React, Remix	Requested; great for a marketing page + app; free on Vercel
Backend framework	NestJS 11	Express, Fastify	Requested; modules, DI, guards and validation built in
Database	PostgreSQL	MongoDB, MySQL	Relational data, transactions, many free hosts
ORM	Prisma 6	TypeORM, raw SQL	Type-safe client, readable schema, simple migrations
Auth	JWT (self-hosted)	Auth0, Clerk, sessions	No third-party account or cost; easy to deploy
File storage	Disk → Cloudinary	AWS S3	Zero setup locally; free tier in production
Versions	Proven majors (Nest 11, Prisma 6, TS 5.9)	Newest majors	Stability over novelty for a project being handed over
AI features	Removed	Keep with mock mode	Owner’s decision; simpler product, no API costs

Challenges & how they were solved

npm 11 blocked Prisma’s install scripts

Added an allowScripts list to package.json so a fresh install generates the Prisma client.

JWT secret read before .env loaded

Switched to JwtModule.registerAsync so config loads first, so tokens always verify.

Port 3000 already used on the dev machine

Moved the frontend to 3100 and updated CORS defaults and docs.

Dates shifting across time zones

Date-only values stored at local noon with shared helpers for every form.

Groceries detected as “subscriptions”

Recurring = one charge a month within ±5%, excluding everyday categories.

Render’s free disk is wiped on restart

Storage abstraction: local disk in dev, Cloudinary when CLOUDINARY_URL is set.

Prisma won’t run destructive migrations non-interactively

Generated SQL with migrate diff, reviewed it, applied with migrate deploy.

Removing AI without breaking the app

Deleted isolated modules, dropped tables by migration, re-ran all browser tests.

Testing & quality checks

Type checking	Strict TypeScript on both apps; every change type-checked before running.
API smoke tests	Real HTTP calls for login, CRUD, analytics, binder, validation (400) and auth (401).
Browser end-to-end	12 user flows automated in Chromium: register → tasks → documents → receipts → binder → settings → logout. Zero console errors.
Visual review	Screenshots of every page at desktop and phone widths, reviewed for layout issues.
Clean-install test	Unzipped the deliverable into an empty folder, installed, migrated a new database, seeded and built.

Running & deploying

Local development setup and the free hosting plan

Run it locally in three steps

1 Database

```
docker compose up -d
```

Postgres 16 on port 5434

2 Backend

```
cd backend
cp .env.example .env
npm ci
npx prisma migrate deploy
npm run db:seed
npm run start:dev
```

API on http://localhost:4000/api

3 Frontend

```
cd frontend
cp .env.example .env.local
npm ci
npm run dev
```

App on http://localhost:3100

Prerequisites: Node.js 20+ and Docker Desktop. Demo login: demo@homevault.app / demo1234

Free hosting plan

Vercel

Frontend (Next.js)

Hobby plan: free, HTTPS, deploys from GitHub

Non-commercial use only

Render

Backend (NestJS)

Free web service + render.yaml blueprint

Sleeps after ~15 min idle; 30–60 s cold start

Neon

PostgreSQL

Free serverless Postgres (~0.5 GB)

Use the direct (non-pooled) URL for migrations

Cloudinary

Uploaded files

Free tier for images & PDFs

Needed because Render's disk is temporary

Browser → Vercel (UI) → Render (API) → Neon (data) + Cloudinary (files). Total cost: $0 for a demo. Free-tier limits change, so check each provider before launch.

Deployment step by step

1

Push to GitHub

Create a repo and push the project (both folders + render.yaml).

2

Create the database

Neon → new project → copy the direct connection string (…?sslmode=require).

3

Set up file storage

Cloudinary → copy the CLOUDINARY_URL from the dashboard.

4

Deploy the API

Render → New → Blueprint → pick the repo; fill DATABASE_URL, FRONTEND_URL, PUBLIC_API_URL, CLOUDINARY_URL. Migrations run on start.

5

Deploy the frontend

Vercel → import repo → root directory “frontend” → set NEXT_PUBLIC_API_URL to the Render URL.

6

Connect & verify

Make FRONTEND_URL on Render match the Vercel URL exactly; open /api/health, then sign up.

Environment variables

Variable	Where	Local value	Production value
DATABASE_URL	Backend	postgresql://homevault:homevault@localhost:5434/homevault	Neon connection string
JWT_SECRET	Backend	any string	Long random string (auto-generated by blueprint)
JWT_EXPIRES_IN	Backend	7d	7d
FRONTEND_URL	Backend	http://localhost:3100	https://<app>.vercel.app
PUBLIC_API_URL	Backend	http://localhost:4000	https://<api>.onrender.com
CLOUDINARY_URL	Backend	(empty → local disk)	cloudinary://key:secret@cloud
PORT	Backend	4000	Set by Render
NEXT_PUBLIC_API_URL	Frontend	http://localhost:4000	https://<api>.onrender.com

Real secrets live only in each host's settings, never in the repository. .env files are git-ignored and excluded from the zip.

Known limitations & next steps

Current limitations

- Billing not connected: plan changes are free
- Uploaded files use unguessable but public URLs
- No email invites or password reset yet
- Money stored as Float, not Decimal
- Free-tier cold starts on Render

Suggested next steps

- Stripe billing and plan-based feature limits
- Private file delivery (signed URLs)
- Email invites, password reset, reminder emails
- Automated test suite in CI (GitHub Actions)
- Rental property P&L module from the reference site

None of these block a local demo or a free deployment; they are the natural next iteration once real users arrive.

Ready to build & deploy

Everything is in homevault.zip. Start with README.md for local setup and the free deployment guide.